

Efficient Computation of the Greatest Common Divisor Using the Euclidean Division Algorithm

Name: Abhijit Santra

Roll No.: CRS2502

Discussion

The Greatest Common Divisor (GCD) of two positive integers is computed using the **Iterative Euclidean Division Algorithm**, one of the oldest and most efficient algorithms in computational mathematics. The algorithm is based on the mathematical property

$$\gcd(a, b) = \gcd(b, a \bmod b), \quad b \neq 0.$$

Instead of checking every possible divisor, the algorithm repeatedly replaces the pair (a, b) by $(b, a \bmod b)$ until the second number becomes zero. The remaining non-zero value is the Greatest Common Divisor. Since each iteration significantly reduces the size of the operands, the algorithm executes very quickly even for large integers.

The implementation is iterative rather than recursive, thereby eliminating recursive function-call overhead and requiring only constant auxiliary memory. The program also measures the processor clock ticks consumed during execution using the standard C library function `clock()`. Although the reported value is not the exact hardware CPU cycle count, it provides a practical estimate of the execution cost. The actual number of hardware cycles depends upon processor architecture, compiler optimizations, cache memory, instruction pipelining and CPU frequency.

Modern arithmetic algorithms also establish that integer multiplication and division possess the same asymptotic time complexity. Classical algorithms perform both operations in $O(L^2)$ bit operations, whereas advanced algorithms such as Karatsuba, Toom–Cook and Schönhage–Strassen achieve nearly identical asymptotic complexities. Consequently, repeated division in the Euclidean algorithm remains computationally efficient.

Algorithm Used

Iterative Euclidean Division Algorithm

1. Read two integers a and b .
2. While $b \neq 0$
 - (a) Compute the remainder $r = a \bmod b$.

- (b) Assign $a = b$.
 - (c) Assign $b = r$.
3. When $b = 0$, output a as the Greatest Common Divisor.
 4. Measure the processor clock ticks required for execution.

C Program

```
1 #include <stdio.h>
2 #include <time.h>
3
4 int gcd(int a, int b)
5 {
6     while (b != 0)
7     {
8         int r = a % b;
9         a = b;
10        b = r;
11    }
12    return a;
13 }
14
15 int main()
16 {
17     int a, b, ans;
18     clock_t start, end;
19
20     printf("Enter two integers: ");
21     scanf("%d %d", &a, &b);
22
23     start = clock();
24
25     ans = gcd(a, b);
26
27     end = clock();
28
29     printf("\nGreatest Common Divisor = %d\n", ans);
30     printf("Clock ticks consumed = %ld\n", (long)(end - start));
31
32     return 0;
33 }
```

Complexity Analysis

The algorithm computes the GCD using repeated Euclidean division. Let

$$n = \max(a, b),$$

where a and b are the input integers.

- **Algorithm:** Iterative Euclidean Division Algorithm.
- **Worst-case Time Complexity:** The running time is

$$O(\log n).$$

In every iteration, the algorithm replaces the larger number by the remainder obtained after division. Since the remainder is always smaller than the divisor, the magnitude of the operands decreases rapidly after each iteration instead of decreasing one by one. Consequently, the number of iterations grows logarithmically with the size of the input, making the algorithm significantly faster than linear approaches.

- **Space Complexity:**

$$O(1).$$

The implementation stores only a fixed number of variables (`a`, `b`, `r`, `ans`, `start` and `end`). No recursion, arrays or dynamically allocated memory are used. Therefore, the memory requirement remains constant irrespective of the input size.

- **Input Length:**

Algorithmic complexity is measured with respect to the number of bits required to represent the input rather than its numerical value. If the larger input integer is n , then the input length is

$$L = \lceil \log_2(n) \rceil$$

bits.

Hence, an integer such as 1024 requires 11 bits for its binary representation, whereas 100 requires only 7 bits. Since

$$L = \log_2(n),$$

the running time can also be expressed as linear in the input length. Thus, the Euclidean algorithm remains highly efficient even when the numerical values of the inputs become extremely large.

References

1. Donald E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, Third Edition, Addison–Wesley, 1997.
2. GeeksforGeeks, “Euclidean Algorithms (Basic and Extended).”
<https://www.geeksforgeeks.org/euclidean-algorithms-basic-and-extended/>
3. OpenAI, *ChatGPT*.
<https://chat.openai.com>